Pokročilé objektové technológie



KST Katedra softvérových technológií
Fakulta riadenia a informatiky, Žilinská univerzita v Žiline

Asynchrónne programovanie II

V minulom dieli ste videli

- Blokovanie IO
  - Thready
  - Asynchrónne programovanie
- Možnosti v C#
  - APM – Asynchronous Programming Model
  - EAP???
  - TAP – Task-based Asynchronous Pattern
- async/await
  - State machine
- Synchronizačný kontext
  - ConfigureAwait(false)

async void

- Dva druhy návratových hodnôt z async metódy:
  - Awaitable (Task, Task<T>, ValueTask<T>)
  - void

```
async Task DoIt()
```

VS

```
async void DoIt()
```

FIGHT

async void

- Once Upon A Time At Google:
  - Why exactly is async void considered bad practice? (SO)
  - Why is async void bad? (redit)
  - The problem with “async void” (blog)
  - Stop using async void in C#! Do this instead. (YT)
  - async void – How to Tame the Asynchronous Nightmare (clanok)

Ako funguje async void

- Živá ukážka

U08
U08a

- Problémy async void
  - Klient sa nedozvie, že/či metóda skončila (nemá Task)
  - Neexistuje možnosť zachytiť výnimku

Neexistuje možnosť zachytiť výnimku

U08b
U09

- Teda...
- Ak nie je SC:
  - AppDomain.CurrentDomain.UnhandledException
  - A padne aplikácia!!!
- Ak je SC:
  - WPF:
    - Application.DispatcherUnhandledException
  - WinForms:
    - Application.ThreadException (nastavit exception mode)
  - ASP .NET MVC
    - override HttpApplication.Application_Error, alebo Controller.OnException

async void sa nedá vyhnúť

1. Typ návratovej hodnoty void predpísaný interfacom
2. Typ návratovej hodnoty void predpísaný delegátom
3. Eventy !!!

async void sa niekedy zíde

- Spúšťanie taskov na pozadí
  - Nezabudnúť try-catch-all
- Možnosť nahradiť za discard operator
  - Výnimku už nezachytí nikto (ale nezhodí aplikáciu)
    - event TaskScheduler.UnobservedTaskException – nespolahlivy
  - Popr. task odložiť na čas, kedy bude spracovateľný

```
_ = DoItAsync();

private static async Task DoItAsync()
{

    ...
}
```


KST Katedra softvérových technológií
Fakulta riadenia a informatiky, Žilinská univerzita v Žiline

Moje riešenie

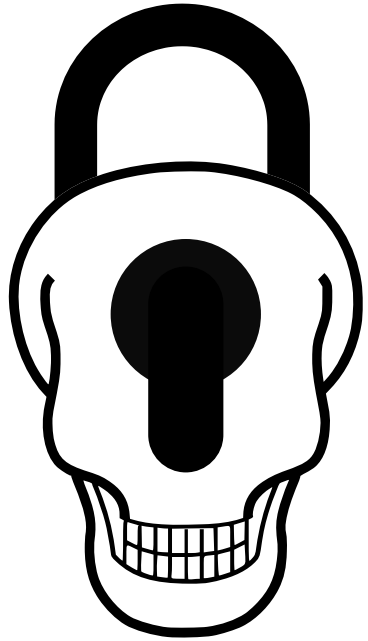
- Nuget balíček
- Inteface IAsyncStarter (KST.AsyncVoid.Abstractions)
  - metóda Start(task)
- Implementácie
  - BackgroundAsyncStarter – spúšťa task ako async void, všetky exception loguje (KST.AsyncVoid)
  - LocalAsyncStarter – pamätá si zoznam všetkých spustených taskov, umožňuje počkať na ich dokončenie a vyhodiť prípadné výnimky (KST.AsyncVoid.Testing)
  - services.AddAsyncVoid() – BackgroundAsyncStarter s logovaním do Microsoft.Extensions.Logging (KST.AsyncVoid.AspNetCore)

Synchrónne vykonanie Task

U10
U10a
U10b
U10c

- Task.Wait() / Task.Result
- Task.GetAwaiter().GetResult()
  - ~~Odporúčaná možnosť~~

- Nerobte to!
  - Ani jedno
  - Blokuje thread!
  - Dead lock



Paralelné spúšťanie taskov

U11
U11a
U11b

- Task.WhenAll
- Čaká na dokončenie všetkých taskov
- Výsledný Task je ukončený, keď sú ukončené všetky

```
await Task.WhenAll(
    Step1(),
    Step2(),
    Step3()
);
```

Rušenie taskov

U12
U12a
U12b
U13

- Podpora v TAP
- Kooperatívne rušenie
  - Task musí počítať s možnosťou rušenia
- Parameter CancellationToken ct
- Vytvorenie pomocou CancellationTokenSource
- Overload s CT takmer vo všetkých BCL async metódach
  - OperationCanceledException

Vlastné ošetrenie CancellationToken

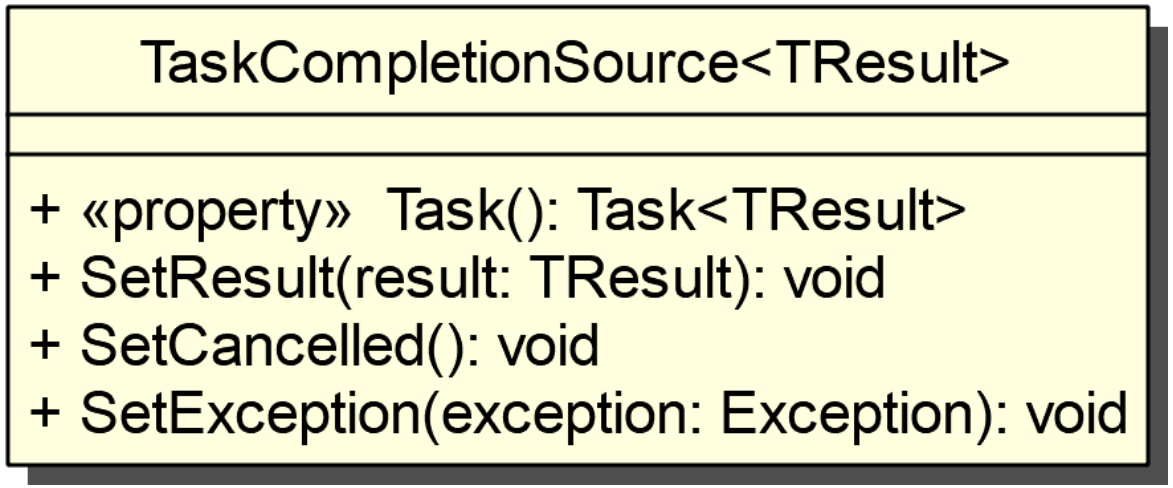
U13

- CancellationToken.IsCancellationRequested
  - True, ak niekto vyžiadal rušenie cez daný CT
- CancellationToken.ThrowIfCancellationRequested
  - OperationCanceledException
- Nutné volať v cykle

- CancellationToken.Register
  - Zaregistruje callback, ktorý sa vykoná, keď niekto vyžiada rušenie cez daný CT

Vytváranie taskov

- Trieda TaskCompletionSource
  - Vytvára tasky
  - Možnosť nastaviť tasku stav



Task.Run

U15
U15a

- Vytváranie taskov pre CPU-bound úlohy
  - Úloha sa vykoná v inom vlákne
  - Vlákno sa na ten čas uvolní

- Hlavne: uvoľnenie GUI (WPF/WinForm)
  - „Nezamrzne“

Na Task.Run pozor

- Používať iba v GUI kóde – mimo nemá zmysel
- Obsadzuje thread z threadpoolu
  - Možnosť nahradiť za priame volanie Task.Factory.StartNew s flagom TaskCreationOptions.LongRunning
- Na nete veľa nezmyselných príkladov
  - Task.Run(semaphore.WaitOne) – zablokovanie vlákna len na čakanie na semafór?
  - Task.Run miesto async void – ??????
  - Task.Run(() => Thread.Sleep(2000)) – joj
  - ....

Async Task.Run

- Task.Run ma overload s Func<Task>
- Spustí aj synchónnu časť async metódy v inom vlákne
- Výsledný Task reprezentuje dokončenie async metódy

Problémy async-await

- Debugovanie
  - Krokovanie môže na await prejsť do iného kódu
  - Debuger nevie zastaviť pri exception
  - Príšerný stack-trace – exception demystifier
- Virálnosť
  - Ak interface vyžaduje, aj synchrónna metóda musí vracať Task
- Nie všetky metódy môžu byť async Task
  - Ak vyžaduje interface
  - Konštruktory

Synchrónne Task metódy

- Vracia Task
- Nevykonáva žiadne asynchrónne operácie

- Nemala by vyhadzovať výnimky
  - `return Task.FromException(...);` miesto `throw ...;`

- Vždy vracia Task
  - `return Task.CompletedTask;` miesto `return;`
  - `return Task.FromResult(...);` miesto `return ...;`

Synchrónne void => Task metódy

```
void WriteString(string text)
{
    Console.WriteLine($"TEXT: {text}");
}
```



```
Task WriteString(string text)
{
    Console.WriteLine($"TEXT: {text}");
    return Task.CompletedTask;
}
```

Synchrónne ret => Task metódy

```
int Compute(int x)
{
    return x + 5;
}
```



```
Task<int> Compute(int x)
{
    return Task.FromResult(x + 5);
}
```


KST Katedra softvérových technológií
Fakulta riadenia a informatiky, Žilinská univerzita v Žiline

Synchrónne return => Task metódy

```
void DoSomethingEvil()
{
    throw new InvalidOperationException("No, I won't");
}
```



```
Task DoSomethingEvilAsync()
{
    return Task.FromException(
        new InvalidOperationException("No, I won‘t“)
    );
}
```

Async konštruktory

- Nedá sa
- Na internete veľa vadných riešení

- Rozumnejšie:
  - Factory method
  - Abstract factory – pri kombinácii s DI
  - Lazy Task

C# 8 a .NET Core 3.x

- Async streams
  - IAsyncDisposable
  - IAsyncEnumerable

- ValueTask

jan.janech@uniza.sk

VĎAKA ZA POZORNOSŤ